EXPERIMENT NO1: Python os.fork and Windows subprocess

Task: T3

Name: Alby Mathew Biju

Roll Number:3

1. Aim

To write a C program that creates a child process with fork(), replaces the child's image with another program using exec, and makes the parent wait for it with wait(). The program shows the parent and child process IDs (getpid, getppid), the wait status, and the child seen with pgrep -a fork_demo while it sleeps. The experiment also explains what each return value of fork() means.

2. Course outcome and module

CO1: Configure and manage system protection tools and evaluate their ability to defend against threats.
Module: Foundation.

3. Tools and versions

Tool	Version / details	Operating system
Python	3.8 or later; record the exact version shown by ‘python3 --version‘ or ‘py --version‘	Ubuntu / macOS / Windows 10 or 11

4. Program or commands

Program A: Linux/macOS — os.fork()

```
import os

print(f”Starting process: PID={os.getpid()}”, flush=True)

try:
pid = os.fork()
except AttributeError:
print(”os.fork() is not available on this operating system.”)
```

```
else:
if pid == 0:
# Child process
print(
f"Child process: PID={os.getpid()}, "
f"Parent PID={os.getppid()}",
flush=True
)
os._exit(0)
else:
# Parent process
print(f"Parent process: PID={os.getpid()}, Child PID={pid}", flush=True)
os.waitpid(pid, 0)
print("Parent: child process has finished.", flush=True)
```

Program B: Windows — subprocess

```
import os
import subprocess
import sys

print(f"Parent process: PID={os.getpid()}", flush=True)

child_code = (
"import os; "
"print(f'Child process: PID={os.getpid()}, Parent PID={os.getppid()}')"
)

result = subprocess.run(
[sys.executable, "-c", child_code],
capture_output=True,
text=True,
check=True
)

print(result.stdout.strip())
print(f"Child process completed with exit code: {result.returncode}")
```

5. How to run

A. Ubuntu/Linux or macOS

1. Open a terminal and change to the folder containing fork_demo.py.
2. Check the installed Python version: python3 --version.

3. Run the program: python3 fork_demo.py.
4. Observe the starting PID, child PID, parent PID, and the message confirming that the child has finished.
5. Capture a screenshot of the terminal output. Include Roll No. 3 in the terminal prompt, a visible label, or the window title as required by the lab rules.

B. Windows 10/11

6. Open Command Prompt or PowerShell in the folder containing subprocess_demo.py.
7. Check the installed Python version: py --version. If the Python launcher is unavailable, use python --version.
8. Run the program: py subprocess_demo.py. Alternatively, run python subprocess_demo.py.
9. Observe the parent PID, child PID, parent PID reported by the child, and the child exit code.
10. Capture a screenshot of the terminal output. Include Roll No. 3 in the terminal prompt, a visible label, or the window title as required by the lab rules.

Important: Do not run fork_demo.py on native Windows; os.fork() is not supported there. Use the subprocess program on Windows.

6. Sample output with screenshots

Linux/macOS:

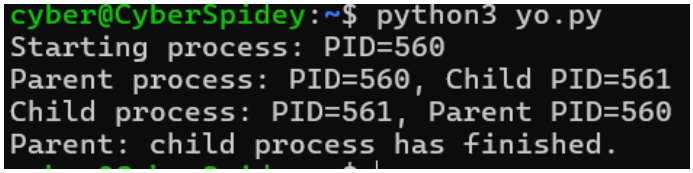

Figure T3-1: Execution of os.fork() program on Linux/macOS.

Windows:

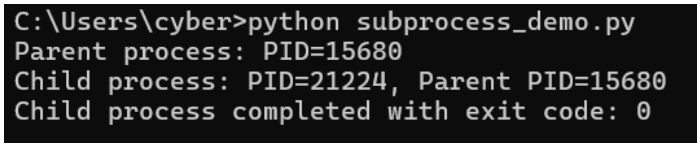

Figure T3-2: Execution of subprocess program on Windows.

7. Result

Process creation was demonstrated using os.fork() on a supported Unix-like operating system and subprocess on Windows. The output showed process identifiers and the parent-child relationship. The parent waited for the child to finish in the fork example, while subprocess.run() waited for the child command to complete on Windows.

8. Difference from other versions of the same experiment

Feature	**os.fork() version**	**Windows subprocess version**
Platform	Unix-like systems such as Linux/macOS	Windows and also available on Unix-like systems
Method	Duplicates the calling process	Starts a child process by running a command/program
Return/identification	Child receives 0; parent receives child PID	CompletedProcess provides return code and captured output
Waiting	os.waitpid() waits for the child	subprocess.run() waits for completion by default
Windows support	Not supported on native Windows	Supported

9. Viva: five questions with answers

Q. What is a process?

Ans. A process is a program in execution, with its own process ID and operating-system-managed resources.

Q. What does os.fork() do?

Ans. On supported Unix-like systems, it creates a child process by duplicating the calling process. It returns 0 in the child and the child PID in the parent.

Q. Why does os.fork() not work on native Windows?

Ans. The Windows process model does not provide the Unix fork() system call through Python's os module. subprocess is a portable alternative for starting another program.

Q. What is the purpose of subprocess.run()?

Ans. It runs a command or program, waits for it to finish, and returns a CompletedProcess object containing information such as the exit code and captured output.

Q. What are PID and PPID?

Ans. PID is the process ID of a process. PPID is the process ID of its parent process.

10. Common errors: message, cause, fix

Message / symptom	Cause	Fix
AttributeError: module 'os' has no attribute 'fork'	The program is being run on native Windows or another platform without os.fork().	Run fork_demo.py on Linux/macOS; use subprocess_demo.py on Windows.
python3 / py is not recognized	The command name or Python launcher differs on that system.	Use the Python command available on the machine, such as python --version and python subprocess_demo.py.
can't open file ...	The terminal is in the wrong folder or the filename is misspelled.	Change to the script's folder and verify the exact filename.
SyntaxError	A line was copied incorrectly or the Python version is too old.	Copy the code carefully and use Python 3.8 or later.
No child output appears	Output buffering or an early process exit can hide output.	Keep flush=True for fork output; for subprocess, capture and print result.stdout as shown.

Sources and declaration

Sources:

- Python documentation — os.fork(): https://docs.python.org/3/library/os.html#os.fork ;
- Python documentation — subprocess: https://docs.python.org/3/library/subprocess.html

AI-use declaration: AI tools were used to help understand and prepare the program code. The code was reviewed and tested as part of the experiment.